

Group 76 Progress Report: Classification of Retinal Diseases Using Optical Coherence Tomography (OCT) Images

Elite Lu, Ishpreet Nagi, Kenneth Ong
{lue13,nagii,salimk4}@mcmaster.ca

1 Introduction

Healthcare providers deal with many vision related illnesses such as Diabetic Macular Edema (DME), Choroidal Neovascularization (CNV), and drusen. They utilize retinal scans to detect and determine these conditions. An issue that arises is the uniqueness of each ailment, as each condition varies in subtle ways. Correctly identifying retinal diseases can prove to be a challenge in a plethora of cases and with approximately 30 million OCT scans performed each year, this process can be time-consuming and labor-intensive for healthcare professionals. By having a classification model that can help speed up this process, healthcare professionals should be able to focus their attention on helping more individuals requiring their aid.

2 Related Work

With the rising popularity of Machine Learning (ML) and Deep Learning (DL) in recent years, it is not surprising that many researchers around the world have worked on the classification of retinal diseases using ML and/or DL methods. Most related works have approached the problem by incorporating transfer learning to perform multiclass or binary classification [Kermany et al., 2018, Miladinović et al., 2024, Reddy, 2025].

As seen in the notebook created by the Kaggle user Badduri Nagendra Reddy [2025], the fine-tuned model based on ResNet50 was able to achieve an impressive 98.97% final test accuracy on the same dataset as the one used in this project. Similarly, the models defined in the papers typically build upon a pre-trained model, where the authors only unfreeze the weights of the last few layers and add a final classification layer [Kermany et al., 2018, Miladinović et al., 2024].

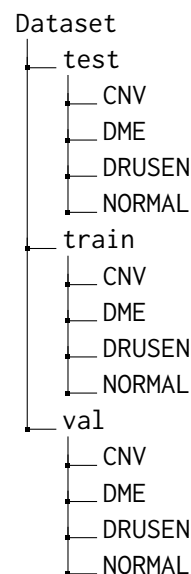
Additionally, these studies experimented with different base models such as VGG16, InceptionV3, and other variations of ResNet, all of which were

able to achieve mid-to-high 90% accuracy [Das et al., 2019, Kermany et al., 2018, Miladinović et al., 2024]. In contrast, the paper by Özdaş et al. [2023] outlined an interesting approach using the Firefly algorithm combined with transfer learning models to extract features from images, followed by classical ML classifiers for binary classification, achieving 95.7% accuracy on their dataset.

3 Dataset

3.1 Folder Structure

When downloading the data, the dataset was partitioned as follows:



3.2 Total Image Counts

Type	Disease	# of Images
train	CNV	37205
	DME	11348
	DRUSEN	8616
	NORMAL	26315
test	CNV	242
	DME	242
	DRUSEN	242
	NORMAL	242
val	CNV	8
	DME	8
	DRUSEN	8
	NORMAL	8

3.3 Preprocessing

Each image was loaded from the various folders as a PIL (Pillow, not the Python Imaging Library) Image and then converted to a tensor object. The pipeline varies slightly for the *train* dataset when compared to both *test* and *val* datasets. Both pipelines resized the images to 224 pixels by 224 pixels and normalized them with means [0.485, 0.456, 0.406] and standard deviation (std) of [0.229, 0.224, 0.225]. Since we are using ResNet50 as the base model, the values for the mean and standard deviation come from ImageNet's preset values, which is the dataset that ResNet is being trained on. The only difference in the preprocessing pipeline is that before normalization, the *train* dataset images might be horizontally flipped whereas the image data for the *TV* pipeline is not. This allowed for the training set to have a more robust dataset. However, this is technically not considered an augmentation since no new data was created (no data distribution changes).

3.4 Annotation

Each image was labeled with an integer to represent its corresponding disease. The images were already named as DISEASE-#####-.jpeg and located in their corresponding folders, which allowed for ease of labeling. When assigning labels to the dataset, we use the following mapping:

CNV	0
DME	1
DRUSEN	2
NORMAL	3

4 Features

4.1 Data Input

Since this is a computer vision/image classification task, the features were derived from the pixels of the images. Each pixel was denoted by three values: Red(R), Green(G), and Blue(B). As mentioned previously, each image was cropped and normalized with values corresponding the means and standard deviations for images from ImageNet.

4.2 Features

The selection of features was not done directly. This was because it was not required as there are no features to select. Since the pixels are the "feature", this would not be justified. As a result, feature engineering and feature selection was not needed. The same can be said for learning embeddings, as this is not a natural language processing task.

For the output features, there are four main features, which are the four categories of diseases: CNV, DME, DRUSEN, and NORMAL.

5 Implementation

5.1 Learning Model

Instead of training a model from scratch, a pre-trained neural network was used and adapted it to the use case. The pretrained neural network selected was ResNet-50, which was trained on the ImageNet dataset. ResNet-50 is a neural network that is 50 layers deep originally developed by Microsoft Research in 2015. This is a type of convolutional neural network (CNN), which is a subset of neural networks that are used for image recognition.

5.2 Implementation Steps

5.2.1 Initializing the Neural Network

To implement the ResNet-50 based neural network, the first step was to instantiate a neural network that uses ResNet-50. This occurs after all the images have been preprocessed. This was called CustomResNet. For the initialization, the input features and classifiers are all defined. The initialization was very similar to typical setup for neural networks.

5.2.2 Freezing and Unfreezing Layers

Upon initialization, the first thing was to freeze all the layers but the last layer, the classifier layer. This was because a pretrained model such as ResNet 50

already comes with various useful general features such as edges, textures, and colours in early layers. Since the dataset that was used was quite small (four disease classes), training all the layers can result in an increase of time, overfitting, and wiping out pretrained knowledge.

5.2.3 Defining the Criterion and Optimizer

Before compiling the model, two additional items need to be addressed, which are the criterion and optimizer. For the criterion, Cross Entropy was selected from the `torch.nn` (neural network) library. For the optimizer, Adam (Adaptive Moment Estimation) from the `torch.optim` library was used. The Adam stochastic optimization method uses two key concepts, which are Momentum and Root Mean Square Propagation (RMSprop), combining them to balance the optimization process. The learning rate defined in Adam was 0.001, a common value for machine learning. The optimization was performed only on the parameters of the final layer since the other layers are currently frozen.

5.2.4 Training Data

The dataset has 83484 images in the four folders (CNV, DME, DRUSEN, NORMAL) under training. The distribution was not even, as CNV had significantly more images compared to the other diseases. For this milestone, 1000 random photos from each folder of the dataset was selected, resulting in an even spread for each disease. This totaled 4000 images for model training for this milestone. It is planned that all the photos will be used alongside additional augmentation for the categories with fewer images in the final model to prevent skewing and bias.

5.2.5 Validation and Test Data

Since there are significantly fewer images categorized under these, all images were used. There are only 32 images for validation spread evenly in the four diseases. The testing data has an even spread too, with 242 images for each disease, totaling 968 images.

5.2.6 Training and Validation

For training, a total of 10 epochs were performed for the 4000 selected images. The loss and accuracy of both training and validation were printed along with a graph displaying the loss for training and validation.

5.2.7 Testing

After testing, the values were plotted on a 4×4 confusion matrix. The final accuracy of the model was written above it.

6 Results and Evaluation

6.1 Training/Validation Phase

After the model was initialized, it was fed through the three stages of training, validation, and testing, with the training and validation phases being combined in a singular loop to save computation time as well as allow for better performance metric representation. During the training and validation phase, the model's performance was measured via the computation of training/validation loss and training/validation accuracy at each epoch. This allows insight into the performance trend of the model and its overall journey through the training and validation phase. The rate at which the model learns the trends in the dataset can be witnessed in its ability to better classify the images through the loss and accuracy metrics. The loss method was the cross-entropy loss, which is accessed through the `nn.CrossEntropyLoss()` function that preexists as a part of PyTorch, and the accuracy was calculated via the division of the correct predictions by the model on both the training and validation datasets by the total predictions by the model on the training and validation sets, respectively. In conjunction with this, loss values are stored and calculated for the training and validation steps, and then visualizing the overall trends by plotting them on a line graph to help further understand the model and its performance.

6.2 Test Phase

By this stage, the model has been evaluated once through the validation phase; however, it was evaluated again, this time on the test split of the dataset via the testing phase. For the testing phase, the performance metric of accuracy was used as the evaluation metric of the performance of the model, which was calculated via the same accuracy calculation method as training and validation. In addition to these performance metrics, a confusion matrix was displayed, which visualizes the true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions of the model for the testing phase. This aided in easier comprehension and evaluation of the performance of the model on the testing dataset, thus presenting

better methods of improvement.

6.3 Quantitative Results

The following results are shown below, which will be acting as the baselines going forward.

----- Training and Validation Phase -----	
Epoch [1/10]:	
Training - Train Loss:	1.0887, Train Accuracy: 0.5433
Validation - Val Loss:	0.6531, Val Accuracy: 0.7812
Epoch [2/10]:	
Training - Train Loss:	0.7874, Train Accuracy: 0.7192
Validation - Val Loss:	0.5057, Val Accuracy: 0.8750
Epoch [3/10]:	
Training - Train Loss:	0.6839, Train Accuracy: 0.7508
Validation - Val Loss:	0.4508, Val Accuracy: 0.8438
Epoch [4/10]:	
Training - Train Loss:	0.6904, Train Accuracy: 0.7365
Validation - Val Loss:	0.3884, Val Accuracy: 0.9062
Epoch [5/10]:	
Training - Train Loss:	0.6213, Train Accuracy: 0.7712
Validation - Val Loss:	0.3650, Val Accuracy: 0.9375
Epoch [6/10]:	
Training - Train Loss:	0.6065, Train Accuracy: 0.7668
Validation - Val Loss:	0.3354, Val Accuracy: 0.9062
Epoch [7/10]:	
Training - Train Loss:	0.5787, Train Accuracy: 0.7800
Validation - Val Loss:	0.3003, Val Accuracy: 0.9062
Epoch [8/10]:	
Training - Train Loss:	0.5600, Train Accuracy: 0.7983
Validation - Val Loss:	0.3019, Val Accuracy: 0.9062
Epoch [9/10]:	
Training - Train Loss:	0.5657, Train Accuracy: 0.7867
Validation - Val Loss:	0.3093, Val Accuracy: 0.9062
Epoch [10/10]:	
Training - Train Loss:	0.5466, Train Accuracy: 0.7920
Validation - Val Loss:	0.2906, Val Accuracy: 0.9062

Figure 1: Training and validation loss and accuracy per epoch.

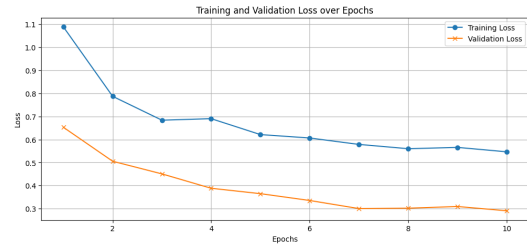


Figure 2: Training and validation loss over epochs.

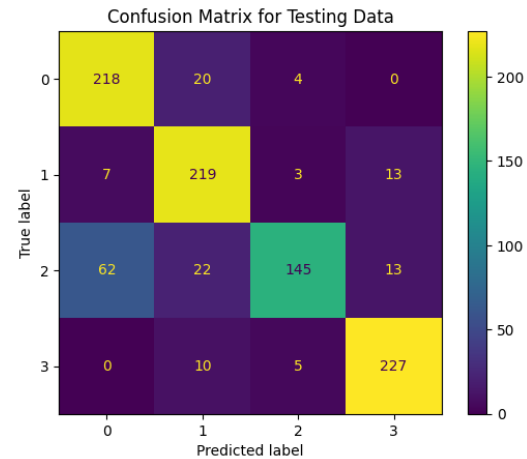


Figure 3: Confusion Matrix of the testing results.

As seen above, while the model was proceeding as intended, there was a large margin for improvement.

7 Feedback and Plans

Looking at the results of the evaluation and metrics, it appears that the model and the results are on the right track. The model appears to have successfully achieved an adequate model baseline that was trained on a small subset of the dataset.

7.1 Image Processing and Augmentation

As mentioned in section 3, some image preprocessing that involved cropping, horizontally flipping, and normalizing was performed on the images. This presents the first point of improvement that could be done. From the discussion with Parisa, it was identified that the distribution of the training data was skewed towards CNV and Normal, which may cause overfitting and bias towards those labels. As a result, augmenting the training data for DME and Drusen so that there was about the same number of data for each category could reduce bias and skew. Building upon this, another improvement is to add more image transformations such as adding jitter and affine transformations to introduce imperfections in the image data, which could help make the model more robust through

adversarial training.

7.2 Additional Metrics

The second point of improvement that can be done would be to add more metrics to help evaluate the results better. In the same conversation, Parisa mentioned that it might be useful to have precision, recall, and F-1 score as a metric in addition to the confusion matrix and accuracy that is currently presented. It would be worthwhile to have the area under the ROC curve (AUC) as a metric for accuracy in tangent with the other metrics mentioned.

7.3 Hyperparameters and Dataset

Once the improvements outlined above are implemented, the next action would be to train the model on a bigger subset of the training dataset (around 5000 data points) to find the best hyperparameters for the model. While training, the current goal is to optimize for the recall and area under the ROC curve for this project. It is required that the classification model minimizes the number of false negatives, which in this case means that patients that actually have the disease should not go undiagnosed.

Finally, a final training run using the best hyperparameters on the entire training and validation dataset is planned along, with an evaluation using the testing dataset at the end proceeding afterwards.

8 Team Contributions

8.1 Elite

- Worked on the initial image opening and storing of data.
- Wrote sections 1, 3, 4, and 5 of the document.

8.2 Ishpreet

- Wrote the model training code and the ResNet-50 neural network.
- Trained and tested the model.
- Wrote section 6 of the document.

8.3 Kenneth

- Worked on setting up the neural network and research for the start of the project.
- Wrote sections 2 and 7 of the document.

References

Amrit Das, Rohan Giri, Gunjan Chourasia, and A. Anileet Bala. 2019. [Classification of retinal diseases using transfer learning approach](#). In *2019 International Conference on Communication and Electronics Systems (ICCES)*, pages 2080–2084.

Daniel S Kermany, Michael Goldbaum, Wenjia Cai, Carolina C S Valentim, Huiying Liang, Sally L Baxter, Alex McKeown, Ge Yang, Xiaokang Wu, Fangbing Yan, Justin Dong, Made K Prasadha, Jacqueline Pei, Magdalene Y L Ting, Jie Zhu, Christina Li, Sierra Hewett, Jason Dong, Ian Ziyar, and 20 others. 2018. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5):1122–1131.e9.

Aleksandar Miladinović, Alessandro Biscontin, Miloš Ajčević, Simone Krešević, Agostino Accardo, Dario Marangoni, Daniele Tognetto, and Leandro Inferrera. 2024. Evaluating deep learning models for classifying OCT images with limited data and noisy labels. *Sci. Rep.*, 14(1):30321.

Badduri Nagendra Reddy. 2025. With causality. <https://www.kaggle.com/code/nagendraredy777/with-casuality>. Accessed: 2025-11-9.

Mehmet Batuhan Özdaş, Fatih Uysal, and Firat Hardalaç. 2023. [Classification of retinal diseases in optical coherence tomography images using artificial intelligence and firefly algorithm](#). *Diagnostics*, 13(3).